Arbeiten mit Git Remotes (dezentrale Repos)

Carsten Gips (HSBI)

TL;DR

Eine der Stärken von Git ist das Arbeiten mit verteilten Repositories. Zu jeder Workingcopy gehört eine Kopie des Repositories, wodurch jederzeit alle Informationen einsehbar sind und auch offline gearbeitet werden kann. Allerdings muss man für die Zusammenarbeit mit anderen Entwicklern die lokalen Repos mit den "entfernten" Repos (auf dem Server oder anderen Entwicklungsrechnern) synchronisieren.

Beim Klonen eines Repositories mit `git clone <url>` wird das fremde Repo mit dem Namen `origin` im lokalen Repo festgehalten. Dieser Name wird auch als Präfix für die Branches in diesem Repo genutzt, d.h. die Branches im Remote-Repo tauchen als `origin/<branch>` im lokalen Repo auf. Diese Remote-Branches kann man nicht direkt bearbeiten, sondern man muss diese Remote-Branches in einem lokalen Branch auschecken und dann darin weiterarbeiten. Es können beliebig viele weitere Remotes dem eigenen Repository hinzugefügt werden.

Änderungen aus einem Remote-Repo können mit `git fetch <remote>` in das lokale Repo geholt werden. Dies aktualisiert **nur** die Remote-Branches `<remote>/<branch>`! Die Änderungen können anschließend mit `git merge <remote>/<branch>` in den aktuell in der Workingcopy ausgecheckten Branch gemergt werden. (*Anmerkung*: Wenn mehrere Personen an einem Branch arbeiten, will man die eigenen Arbeiten in dem Branch vermutlich eher auf den aktuellen Stand des Remote **rebasen** statt mergen!) Eigene Änderungen können mit `git push <remote> <branch>` in das Remote-Repo geschoben werden.

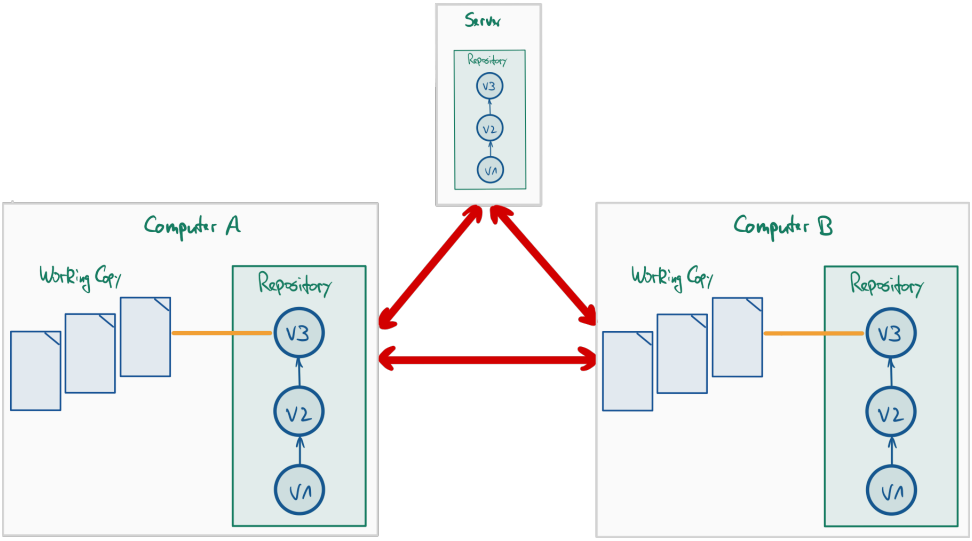
Um den Umgang mit den Remote-Branches und den davon abgeleiteten lokalen Branches zu vereinfachen, gibt es das Konzept der "Tracking Branches". Dabei "folgt" ein lokaler Branch einem Remote-Branch. Ein einfaches `git pull` oder `git push` holt dann Änderungen aus dem Remote-Branch in den ausgecheckten lokalen Branch bzw. schiebt Änderungen im lokalen Branch in den Remote-Branch.

Videos

- VL Git Remotes
- Demo Fetch, Pull und Push
- Demo Tracking-Branches
- Demo Verknüpfen weiterer Remotes

---

1 Nutzung von Git in Projekten: Verteiltes Git (und Workflows)



Git ermöglicht eine einfaches Zusammenarbeit in verteilten Teams. Nachdem wir die verschiedenen Branching-Strategien betrachtet haben, soll im Folgenden die Frage betrachtet werden: **Wie arbeite ich sinnvoll über Git mit anderen Kollegen und Teams zusammen? Welche Modelle haben sich etabliert?**

2 Clonen kann sich lohnen ...

```
https://github.com/Programmiermethoden-CampusMinden/PM-Lecture

---C---D---E  master
```

=> `git clone https://github.com/Programmiermethoden-CampusMinden/PM-Lecture`

```
./PM-Lecture/  (lokaler Rechner)

---C---D---E  master
           ^origin/master
```

Git-Repository mit der URL `<URL-Repo>` in lokalen Ordner `<directory>` auschecken:

- `git clone <URL-Repo> [<directory>]`
- Workingcopy ist automatisch über den Namen `origin` mit dem remote Repo auf dem Server verbunden
- Lokaler Branch `master` ist mit dem remote Branch `origin/master` verbunden ("Tracking Branch", s.u.), der den Stand des `master`-Branches auf dem Server spiegelt

Für die URL sind verschiedene Protokolle möglich, beispielsweise:

- "`file://`" für über das Dateisystem erreichbare Repositories (ohne Server)
- "`https://`" für Repo auf einem Server: Authentifikation mit Username und Passwort (!)

- "`git@`" für Repo auf einem Server: Authentifikation mit **SSH-Key** (diese Variante wird im Praktikum im Zusammenspiel mit dem Gitlab-Server im SW-Labor verwendet)

3 Eigener und entfernter *master* entwickeln sich weiter ...

```
https://github.com/Programmiermethoden-CampusMinden/PM-Lecture

---C---D---E---F---G  master
```

```
./PM-Lecture/  (lokaler Rechner)

---C---D---E---H  master
           ^origin/master
```

Nach dem Auschecken liegen (in diesem Beispiel) drei `master`-Branches vor:

1. Der `master` auf dem Server,
2. der lokale `master`, und
3. die lokale Referenz auf den `master`-Branch auf dem Server: `origin/master`.

Der lokale `master` ist ein normaler Branch und kann durch Commits verändert werden.

Der `master` auf dem Server kann sich ebenfalls ändern, beispielsweise weil jemand anderes seine lokalen Änderungen mit dem Server abgeglichen hat (`git push`, s.u.).

Der Branch `origin/master` lässt sich nicht direkt verändern! Das ist lediglich eine lokale Referenz auf den `master`-Branch auf dem Server und zeigt an, welchen Stand man bei der letzten Synchronisierung hatte. D.h. erst mit dem nächsten Abgleich wird sich dieser Branch ändern (sofern sich der entsprechende Branch auf dem Server verändert hat).

Anmerkung: Dies gilt analog für alle anderen Branches. Allerdings wird nur der `origin/master` beim Clonen automatisch als lokaler Branch ausgecheckt.

Zur Abbildung: Während man lokal arbeitet (Commit `H` auf dem lokalen `master`), kann es passieren, dass sich auch das remote Repo ändert. Im Beispiel wurden dort die beiden Commits `F` und `G` angelegt (durch `git push`, s.u.).

Wichtig: Da in der Zwischenzeit das lokale Repo nicht mit dem Server abgeglichen wurde, zeigt der remote Branch `origin/master` immer noch auf den Commit `E`!

4 Änderungen im Remote holen und Branches zusammenführen

```
https://github.com/Programmiermethoden-CampusMinden/PM-Lecture

---C---D---E---F---G  master
```

=> `git fetch origin`

```
./PM-Lecture/  (lokaler Rechner)

---C---D---E---H  master
            \
             F---G  origin/master
```

4.1 Änderungen auf dem Server mit dem eigenen Repo abgleichen

Mit `git fetch origin` alle Änderungen holen

- Alle remote Branches werden aktualisiert und entsprechen den jeweiligen Branches auf dem Server: Im Beispiel zeigt jetzt `origin/master` ebenso wie der `master` auf dem Server auf den Commit `G`.
- Neue Branches auf dem Server werden ebenfalls "geholt", d.h. sie liegen nach dem Fetch als entsprechende remote Branches vor
- Auf dem Server gelöschte Branches werden nicht automatisch lokal gelöscht; dies kann man mit `git fetch --prune origin` automatisch erreichen

Wichtig: Es werden nur die remote Branches aktualisiert, nicht die lokalen Branches!

4.2 *master*-Branch nach "git fetch origin" zusammenführen

1. Mit `git checkout master` Workingcopy auf eigenen `master` umstellen
2. Mit `git merge origin/master` Änderungen am `origin/master` in eigenen `master` mergen
3. Mit `git push origin master` eigene Änderungen ins remote Repo pushen

```
https://github.com/Programmiermethoden-CampusMinden/PM-Lecture

---C---D---E---H---I  master
            \     /
             F---G
```

```
./PM-Lecture/  (lokaler Rechner)

---C---D---E---H---I  master
            \     /^origin/master
             F---G
```

Anmerkung: Schritt (2) kann man auch per `git pull origin master` erledigen … Ein `pull` fasst `fetch` und `merge` zusammen (s.u.).

Anmerkung Statt dem `merge` in Schritt (2) kann man auch den lokalen `master` auf den aktualisierten `origin/master` rebasen und vermeidet damit die "Raute". Der `pull` kann mit der Option "`--rebase`" auf "rebase" umgestellt werden (per Default wird bei `pull` ein "merge" ausgeführt).

4.3 Auf dem Server ist nur ein *fast forward merge* möglich

Sie können Ihre Änderungen in Ihrem lokalen `master` auch direkt in das remote Repo pushen, solange auf dem Server ein **fast forward merge** möglich ist.

Wenn aber (wie in der Abbildung) der lokale und der remote `master` divergieren, müssen Sie den Merge wie beschrieben lokal durchführen (`fetch`/`merge` oder `pull`) und das Ergebnis wieder in das remote Repo pushen (dann ist ja wieder ein *fast forward merge* möglich, es sei denn, jemand hat den remote `master` in der Zwischenzeit weiter geschoben - dann muss die Aktualisierung erneut durchgeführt werden).

Beispiel für Zusammenführen (merge und push), Anmerkung zu fast forward merge (https://youtu.be/moqywsxtEy8)

5 Branches und Remotes

- Eigenen (neuen) lokalen Branch ins remote Repo schicken
  - `git push <remote> <branch>`

- Neuer Branch im remote Repo
  - `git fetch <remote>` holt (auch) alle neuen Branches
  - Lokale Änderungen an remote Branches nicht möglich!
    => **Remote Branch in lokalen Branch mergen** (oder auschecken)

6 Zusammenfassung: Arbeiten mit Remotes

1. Änderungen vom Server holen: `git fetch <remote>`
   => Holt alle Änderungen vom Repo `<remote>` ins eigene Repo (Workingcopy bleibt unangetastet!)

2. Aktuellen lokalen Branch auffrischen: `git merge <remote>/<branch>`
   (oder alternativ `git pull <remote> <branch>`)

3. Eigene Änderungen hochladen: `git push <remote> <branch>`

6.1 Anmerkung: *push* geht nur, wenn

1. Ziel ein “bare”-Repository ist, **und**
2. keine Konflikte entstehen

=> im remote Repo nur “fast forward”-Merge möglich

=> bei Konflikten erst `fetch` und `merge`, danach `push`

Anmerkung: Ein "bare"-Repository enthält keine Workingcopy, sondern nur das Repo an sich. Die ist bei Repos, die Sie auf einem Server wie Gitlab oder Github anlegen, automatisch der Fall. Sie können aber auch lokal ein solches "bare"-Repo anlegen, indem Sie beim Initialisieren den Schalter `--bare` mitgeben: `git init --bare`...

6.2 Beispiel

```
1 git fetch origin           # alle Änderungen vom Server holen
2 git checkout master        # auf lokalen Master umschalten
3 git merge origin/master    # lokalen Master aktualisieren
4
5 ... # Herumspielen am lokalen Master
6
7 git push origin master     # lokalen Master auf Server schicken
```

7 Vereinfachung: Tracking Branches

- **Tracking Branch**: lokaler Branch, der remote Branch "verfolgt"
  - Beispiel: lokaler `master`-Branch folgt `origin/master` per Default

- **Vereinfachung im Workflow**:
  - `git pull` entspricht
    1. `git fetch <remote>` **plus**
    2. `git merge <remote>/<branch>`
  - `git push` entspricht `git push <remote> <branch>`

Vorsicht: `pull` und `push` beziehen sich nur auf ausgecheckten Tracking Branch

8 Einrichten von Tracking Branches

- `git clone`: lokaler `master` trackt automatisch `origin/master`

- Remote Branch als Tracking Branch einrichten:
  1. Änderungen aus remote Repo holen: `git fetch <remote>`
  2. Tracking Branch anlegen: `git checkout -t <remote>/<branch>` (=> Option `-t` richtet den remote Branch als Tracking Branch ein)

- Lokalen neuen Branch ins remote Repo schicken und als Tracking Branch einrichten:
  1. Lokalen Branch erzeugen: `git checkout -b <branch>`
  2. Lokalen Branch ins Repo schicken: `git push -u <remote> <branch>` (=> Option `-u` richtet den lokalen Branch als Tracking Branch ein)

9 Hinzufügen eines (weiteren) Remote Repository

Sie können einem Repo beliebig viele Remotes hinzufügen:

`git remote add <name> <url>`

Beispiel: `git remote add andi git@github.com:andi/repo.git`

- Remote `origin` wird bei `clone` automatisch angelegt
- Ansehen der Remotes mit `git remote -v`
- `fetch`, `push` und `pull` jeweils über den vergebenen Namen

Beispiel: `git fetch andi` oder `git push origin master`

10 Wrap-Up

- Synchronisierung des lokalen Repos mit anderen Repos
  - Repo kopieren: `git clone <url>`
  - Interner Name fürs fremde Repo: `origin`
  - Änderungen vom fremden Repo holen: `git fetch <remote>`
  - Änderungen in lokalen Branch einpflegen: `git merge <remote>/<branch>`
  - Eigene Änderungen ins fremde Repo schieben: `git push <remote> <branch>`

- Tracking Branches (Konzept, Anwendung)
  - Remote Branches können lokal nicht verändert werden:
    * In lokale Branches mergen, oder
    * Tracking Branches anlegen => einfaches `pull` und `push` nutzen
  - Tracking Branches sind lokale Branches, die remote Branches verfolgen (“tracken”)

11 Zum Nachlesen

- Chacon und Straub (2014, Kap. 3)
- Atlassian Pty Ltd (2022)
- Github Inc. (2022)

Lernziele

- k3: Erzeugen eines Clones von fremden Git-Repositories
- k3: Holen der Änderungen vom fremden Repo
- k3: Aktualisierung der lokalen Branches
- k3: Pushen der lokalen Änderungen ins fremde Repo
- k3: Anlegen von lokalen Branches vs. Anlegen von entfernten Branches
- k3: Anlegen eines Tracking Branches zum Vereinfachen der Arbeit

Quizzes

- Quiz Git Remotes (ILIAS)

Challenges
Interaktive Git-Tutorials: Schaffen Sie die Rätsel?

- Learn Git Branching
- Oh My Git!
- Git Time

Quellen

Atlassian Pty Ltd. 2022. „Become a Git Guru.“ 2022. https://www.atlassian.com/git/tutorials.
Chacon, S., und B. Straub. 2014. *Pro Git*. 2. Aufl. Apress. https://git-scm.com/book/en/v2.
Github Inc. 2022. „Git Cheat Sheets“. 2022. https://training.github.com/.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 9d7ab85 (Remove Hugo: remove pandoc escaping on video links, 2025-04-30)